Prepare answers before the exercise class. In class you will present them.

5.1 Consensus without faults

Implement a simple algorithm that solves the consensus problem in an asynchronous system with n nodes and without faults! You can use ppp (*PerfectPointToPointLink*) links between nodes.

Interface Interface and properties of (regular) consensus

```
Events:
    Request: <Propose, v>: Proposes value v for consensus.
    Indication: <Decide, v>: Outputs a decided value v of consensus.
Properties:
    • Termination: Every correct process eventually decides some
      value.
    • Validity: If a process decides v, then v was proposed by
      some process.
    • Integrity: No process decides twice.
    • Agreement: No two correct processes decide differently.
```

5.2 Bracha reliable broadcast: Lemmas 2–4

Recall Bracha's asynchronous reliable broadcast ($n = 3f + 1$) and Lemma 1 (correct processes send ready for at most one common value).

Prove the following:

1. **Lemma 2:** If two correct processes p and q accept the values v and u, respectively, then $u = v$.
2. **Lemma 3:** If a correct process p accepts the value v, then every other correct process eventually accepts v.
3. **Lemma 4:** If the transmitter is correct and sends v, then all correct processes accept v.

5.3 Atomic broadcast: latency and message complexity

Recall the Défago et al. classification of atomic-broadcast algorithms: sequencer-based (variants UB, UUB, BB), privilege-based / token-based, communication-history, and consensus-based.

Cost model. Assume a failure-free system with n processes and reliable links. One process broadcasts a single payload. A point-to-point send costs **1** latency step and **1** message. A broadcast is implemented as n simultaneous unicasts: it costs **1** latency step and n messages. Latency means the number of communication steps until every process can deliver the payload (best case unless asked otherwise). Ignore consensus-based algorithms here (their cost is that of the underlying consensus).

(a) For each of the sequencer variants **UB**, **UUB**, and **BB**, give the latency (in steps) and the number of messages.

(b) For a **circulating token** (privilege-based) algorithm, give latency and message count in two cases:

  (i) the sender already holds the token;

  (ii) worst case: the token must travel before the sender may broadcast.

  (You may ignore on-demand token request; assume the token circulates continuously among the n processes.)

(c) Consider a **communication-history** algorithm that tags messages with either Lamport clocks or vector clocks (choose one). Argue why the *message* count for disseminating one payload looks like a single broadcast, but the *latency until delivery* has no small constant in a purely asynchronous model.